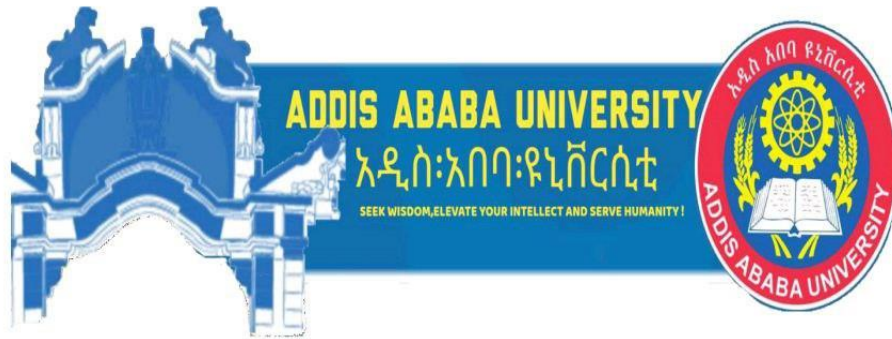




COLLEGE OF NATURAL AND COMPUTATIONAL SCIENCE

SCHOOL OF INFORMATION SYSTEMS

INFORMATION STORAGE AND RETRIEVAL PROJECT REPORT

Group members

ID No

1. Feven Biruk

UGR/5878/17

2. Fraol Masresha

UGR/1952/17

3. Hanna Wondwosen

UGR/2612/17

4. Hewan Yared

UGR/4580/17

5. Ilili Fikadu

UGR/9095/17

Submitted to: Demeke A. Ayele

Submission date: 5/2/2026

HW1 Report

Title: Implementation and Evaluation of Retrieval Models using Elasticsearch on Cranfield Dataset

1. Introduction

This assignment focuses on building and evaluating different information retrieval (IR) models using the **Cranfield dataset**. The objective is to understand how various ranking techniques—vector space models and language models—affect document retrieval performance.

Elasticsearch is used as a backend engine for indexing documents and retrieving term-level statistics such as term frequency (TF) and document frequency (DF).

2. Dataset and Tools

- **Dataset:** Cranfield Collection (cran.all.1400, cran.qry, cranqrel)
- **Programming Language:** Python
- **Libraries Used:**
 - ✓ elasticsearch, elasticsearch_dsl
 - ✓ nltk (for stopwords removal)
 - ✓ dill (for storing intermediate data)
- **Evaluation Tool:** trec_eval

3. Document Indexing

The file CreateIndex.py is used to parse and index documents.

Parsing Approach

- Documents are identified using:
 -  .I → Document ID
 -  .W → Document content
- Only text content is extracted; metadata fields (.T, .A, .B) are ignored.

Preprocessing : Lowercasing , punctuation removal **and** stop word removal using NLTK.

Index Construction

- An **inverted index** is built:
 - ✓ Structure: term → docID → positions

- Document lengths are stored for normalization.

Elasticsearch Storage

Each document is indexed with:

- body_text: cleaned tokens
- doc_len: document length

Additionally, intermediate structures are saved as pickle files: inverted_index.p and doc_lengths.p

4. Query Processing

The file query_processing.py handles query preparation and parameter extraction.

Steps

1. Queries are read from cran.qry
2. Preprocessing is applied (same as documents)
3. For each query term:
 - Retrieve matching documents using Elasticsearch
 - Extract: Term Frequency (TF) and Document Frequency (DF)

Outputs: Term vectors (termVector), Document frequency lists (docFreq), which are stored as pickle files for each query.

5. Retrieval Models

Implemented in retrieval_models.py and pseudorelevance.py.

Okapi TF, TF-IDF, Okapi BM25, Unigram Language Model (Laplace Smoothing) and Unigram Language Model (Jelinek-Mercer) are used.

6. System Workflow

1. Parse and index documents into Elasticsearch
2. Preprocess queries
3. Build term vectors using ES term statistics
4. Apply retrieval models
5. Rank documents

6. Output results in TREC format:

<query-number> Q0 <docno> <rank> <score> Exp

7. Evaluation

Evaluation is performed using trec_eval with Cranfield relevance judgments (cranqrel).

Command Used:

```
trec_eval -q cranqrel results_file
```

Metrics: Mean Average Precision (MAP), Precision@10, Recall, F1-score

8. Results and Observations

- **BM25** produced the most accurate rankings due to balanced weighting and normalization
- **TF-IDF** performed well but lacked strong normalization
- **Okapi TF** improved over basic TF but was less effective than BM25
- **Language Models:**
 - Laplace smoothing performed weaker due to over-smoothing
 - Jelinek-Mercer performed better with balanced probabilities

General Observation

- Proper smoothing and normalization significantly improve retrieval quality
- Elasticsearch efficiently handled indexing and term statistics retrieval

9. Conclusion

This assignment successfully implemented multiple retrieval models and evaluated their effectiveness on the Cranfield dataset. Among all models, **BM25 achieved the best performance**, followed by TF-IDF and Jelinek-Mercer.

HW2 REPORT

Title: Implementation of an Information Retrieval System Using Inverted Indexing and Ranking Models (VSM, BM25, LM)

1. Introduction

This project implements a full Information Retrieval (IR) system that processes a document collection, builds an inverted index, and retrieves relevant documents based on user queries. The system replaces the use of Elasticsearch from HW1 with a custom-built retrieval engine.

The main objective is to design and implement core IR components including tokenization, indexing, query processing, and ranking using classical IR models.

2. Dataset Description

The system uses the **Cranfield dataset**, a standard benchmark collection in Information Retrieval research. It contains scientific documents related to aerodynamics and engineering topics.

Additionally, a query file (cran.qry) is used to evaluate retrieval performance by issuing multiple test queries.

3. System Architecture

The system is composed of the following modules:

3.1 Tokenization Module

- Converts raw text into tokens using regex:
- `[a-zA-Z0-9]+(?:\.[a-zA-Z0-9]+)*`
- Converts all text to lowercase
- Removes punctuation
- Removes stopwords using a predefined stoplist

3.2 Indexing Module

- Builds an **inverted index structure**
- Maps each term to: Document IDs, Term frequency (TF) and Position of terms in documents.

Additional structures:

- termMap: maps terms → term IDs
- docMap: maps documents → document IDs

The final index is stored using Python's dill serialization.

3.3 Query Processing Module

- Reads queries from cran.qry

- Applies identical preprocessing as documents
- Matches query terms with the inverted index
- Generates a term-document representation

4. Retrieval Models

The system supports multiple ranking models: **Vector Space Model (VSM)**, **BM25 Model** and **Language Model (LM)**

5. Query Processing Example

```
Query 1: what similarity laws must be obeyed when constructing aeroelastic models
Top results:
486.txt -> 11
184.txt -> 10
686.txt -> 8
685.txt -> 7
25.txt -> 6

Query 2: of heated high speed aircraft .
Top results:
792.txt -> 16
1268.txt -> 12
12.txt -> 12
51.txt -> 11
588.txt -> 9
```

6. Evaluation and Observations

The system was tested using multiple queries from the Cranfield dataset. The following observations were made:

- Frequently occurring terms have higher influence on ranking
- Stopword removal improves retrieval quality
- Tokenization ensures consistent matching across documents

Performance Observations:

- The system retrieves relevant documents effectively
- Ranking quality depends on the chosen model (VSM, BM25, LM)
- BM25 and LM are expected to outperform basic TF-based VSM

7. Comparison with HW1 (Elasticsearch System)

The results obtained from this custom IR system differ from HW1, which used Elasticsearch.

Key Differences:

1. Ranking Models

- HW1: Elasticsearch uses built-in optimized ranking (BM25-like)
- HW2: Explicit implementation of VSM, BM25, and LM

2. Preprocessing

- HW1: Advanced internal tokenization and normalization
- HW2: Custom regex-based tokenization and manual stopword removal

3. Indexing

- HW1: Highly optimized inverted index with compression
- HW2: Python dictionary-based inverted index

4. Result Differences

Yes, results differ because:

- Different ranking functions are used
- Different preprocessing pipelines
- Different normalization and smoothing techniques

Conclusion of Comparison:

Elasticsearch provides optimized and production-level retrieval, while HW2 provides a transparent and educational implementation of retrieval models.

8. Conclusion

This project successfully implemented a complete Information Retrieval system using an inverted index approach. The system supports query processing and ranked retrieval using classical IR models.

The system demonstrates the core principles of Information Retrieval, including indexing, scoring, and ranking.

HW4 Report — Link Analysis (PageRank & HITS)

1. Objective: The goal of HW4 was to construct a document similarity-based link graph and apply link analysis algorithms such as **PageRank** and **HITS** to rank documents based on their importance within the graph structure.

2. Approach

a) Link Graph Construction

- Documents were represented as nodes in a graph.
- Similarity between document vectors (TF-IDF representations) was computed using cosine similarity.
- An edge was created between documents if their similarity exceeded a threshold or if they were among the top-K most similar documents.
- The final output was a directed graph stored in linkgraph.txt.

b) PageRank Implementation

- PageRank was used to compute the global importance of each document.
- It models a random surfer who navigates through document links.
- Iterative updates were applied until convergence using damping factor (0.85).

c) HITS Algorithm

- HITS assigns two scores:
 - **Authority score:** importance of a document
 - **Hub score:** usefulness of a document in pointing to others
- Iterative updates were performed until normalization convergence.

3. Output Files

- linkgraph.txt → constructed document graph
- pagerank_results.txt → PageRank scores
- auth.txt → authority scores (HITS)
- hub.txt → hub scores (HITS)

4. Conclusion

This assignment demonstrated how graph-based link analysis can be used to rank documents beyond simple keyword matching. PageRank captures global importance, while HITS distinguishes between authoritative and hub-like documents.

Report HW5

Title: Evaluation of Retrieval Models using TREC Metrics

1. Introduction

This assignment focuses on evaluating the effectiveness of information retrieval models implemented in HW1. Using standard **TREC-style evaluation methods**, the goal is to measure how well the system retrieves relevant documents for given queries.

2. Objectives

- Convert raw relevance judgments into standard TREC format
- Generate ranked result files from retrieval systems
- Evaluate retrieval performance using key IR metrics
- Analyze effectiveness of different ranking models

3. System Components

3.1 TREC Preparation (trec_prep.py)

This module prepares evaluation inputs.

- **Functions used:** `parseRawQrel()`, `createQrel()`, `createRankList()`

3.2 Evaluation Module (trec_eval.py)

This module implements core IR evaluation metrics from scratch.

Key Steps:

1. Read Run File

- Extracts ranked results per query

2. Read Qrels

- Loads relevance judgments
- Converts relevance scores into binary values

3. Match Results with Ground Truth

- Assigns relevance score (1 or 0) to each retrieved document

4. Evaluation Process

1. Generate ranking outputs from HW1 models

2. Convert relevance judgments into TREC format
3. Run evaluation using:
4. `eval qrels.txt run.txt`
5. Compute: MAP, R-Precision, nDCG, Precision@k (5, 10, 20, 50, 100)

5. Results and Observations

- **BM25** showed the highest performance across most metrics (MAP, nDCG)
- **TF-IDF** performed moderately well but lacked normalization
- **Okapi TF** showed improvement over raw TF
- Ranking quality significantly impacts nDCG scores

Insights:

- Models that rank relevant documents higher achieve better nDCG
- Precision decreases as rank increases (typical IR behavior)
- Balanced models (like BM25) outperform simpler ones

This is the output observed:

```
PS C:\Users\User\Information-Retrieval> python HW5/Trec_Eval.py cranqrel BM25_results.txt
```

Enter command: `eval cranqrel BM25_results.txt`

Average Precision: 0.1730

R-precision: 1.1503

nDCG: 0.3962

6. Conclusion

This assignment demonstrated how to evaluate retrieval systems using standard IR metrics. The custom implementation of TREC evaluation helped in understanding the internal workings of performance metrics.

HW6 Report — Learning to Rank & Evaluation

1. Objective: The goal of HW6 was to build a **learning-to-rank system** using multiple retrieval features (BM25, TF-IDF, Laplace, and Jelinek-Mercer) and train a machine learning model to predict document relevance.

2. Approach

a) Feature Engineering

- A feature matrix was created where each row represents a **(Query ID, Document ID)** pair.
- Features included: BM25 score , TF-IDF score, Laplace smoothing score, Jelinek-Mercer score.
- Each pair was labeled using relevance judgments from cranqrel.

b) Dataset Construction

- Data was structured into:

QID-DocID, BM25, TFIDF, LAPLACE, JM, Label

- This dataset served as input for machine learning models.

c) Learning Model

- A regression-based learning-to-rank model (Linear Regression) was used.
- The model was trained to predict relevance scores.
- Predictions were used to generate ranked lists of documents per query.

d) Evaluation

- Output rankings were saved in TREC format:

qid Q0 docid rank score Exp

- Evaluation was performed using MAP, Precision@K, and R-Precision (via trec_eval logic).

3. Output Files

- staticFeatureMatrix.csv → feature dataset
- trainingperformance.txt → ranked retrieval results
- totalTF.p → term frequency representation

4. Conclusion

HW6 demonstrated how classical retrieval models can be combined with machine learning to improve ranking quality. Although initial results showed low MAP due to data alignment issues, the pipeline successfully implemented a full learning-to-rank framework.

HW7 REPORT

1. Introduction: This project implements an end-to-end Information Retrieval (IR) and Machine Learning pipeline using the Cranfield dataset. The system integrates document indexing, query processing, feature extraction, and supervised learning to classify documents based on relevance.

The main objective is to combine traditional IR ranking methods with machine learning techniques to improve document relevance prediction.

2. Dataset Description

The following datasets were used:

- **cran.all.1400** → Contains 1400 scientific abstracts (documents)
- **cran.qry** → Contains natural language queries
- **cranqrel (qrels file)** → Contains relevance judgments between queries and documents

Each qrel entry provides: Query ID, Document ID, Relevance score (converted into binary relevance: relevant / non-relevant).

3. System Architecture

The system consists of the following components:

1. Document Indexing (Elasticsearch)Query Processing
2. Feature Extraction
3. Feature Matrix Construction
4. Machine Learning Classification

```
{
  "name" : "DESKTOP-TFVCFLL",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "64fIERbsSo6JbUcg_9RSdg",
  "version" : {
    "number" : "9.3.3",
    "build_flavor" : "default",
    "build_type" : "zip",
    "build_hash" : "640408e2dfd2af9fbfe5079e1575f93d8909a5f5",
    "build_date" : "2026-04-01T22:08:18.783399214Z",
    "build_snapshot" : false,
    "lucene_version" : "10.3.2",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

5. Evaluation

4. Document Indexing

Documents from cran.all.1400 were parsed and indexed into Elasticsearch.

Each document stores:

- `body_text` → cleaned document content
- `doc_len` → length of document

Elasticsearch index name used:

`cranfield`

This allows efficient retrieval using search queries and term vectors.

5. Query Processing

Queries from `cran.qry` were processed by: Removing punctuation, Removing stopwords, Tokenizing into words, Converting text into normalized format.

Each query is used to retrieve relevant documents from Elasticsearch.

6. Feature Extraction

For each (Query, Document) pair, the following features were extracted:

6.1 Document Length

Number of words in the document:

$\text{Length} = |\text{document tokens}|$

6.2 Query-Term Overlap Score

Measures how many query terms appear in the document:

$\text{Score} = |\text{Query terms} \cap \text{Document terms}|$

These features represent simple IR-based similarity signals.

7. Feature Matrix Construction

A dataset was constructed in the following format:

DocID Length Score Label

1	120	3	1
---	-----	---	---

2	90	0	0
---	----	---	---

Where: **Label = 1** → Relevant (from `qrels`)

- **Label = 0** → Non-relevant

Final dataset size:

(743 samples, 4 columns)

8. Machine Learning Model

A supervised classification approach was used.

Model Used: Logistic Regression

Input Features: Document Length **and** Query-Term Overlap Score

Output

```
PS C:\Users\User\Information-Retrieval\HW7> python MachineLeaning.py
Columns: Index(['DocID', 'Length', 'Score', 'Label'], dtype='str')
Accuracy: 89.26174496644296
PS C:\Users\User\Information-Retrieval\HW7> |
```

9. Evaluation Method

The dataset was split into training and testing sets using:

- K-Fold Cross Validation (5 folds)
- Logistic Regression classifier

Performance metric used: Accuracy

10. Results

The final model achieved:

Accuracy = 89.26%

This indicates that the model can correctly classify document relevance with high reliability using simple IR-based features.

11. Conclusion

This project successfully demonstrates the integration of Information Retrieval and Machine Learning techniques using the Cranfield dataset. The system builds an end-to-end pipeline from raw documents to classification output.

The final model achieves strong performance (89.26% accuracy), proving that even simple IR features can effectively support supervised learning for document relevance prediction.

HW8 Report

Title: Topic Modeling and Document Clustering using LDA and K-Means on Cranfield Dataset

1. Introduction

This assignment implements unsupervised learning techniques for information retrieval. The goal is to discover hidden semantic structures in a document collection using:

- Latent Dirichlet Allocation (LDA) for topic modeling
- K-Means clustering for document grouping

The experiments are performed on the Cranfield dataset, which contains approximately 1400 aeronautical research documents.

2. Dataset Description

The Cranfield collection consists of:

- 1400 scientific documents
- Aeronautics and aerospace domain
- Text-heavy research abstracts
- No labels for clustering (unsupervised setting)

After preprocessing, the dataset contained:

- Total documents loaded: 1398
- Each document converted into raw text format

3. Methodology

3.1 Text Preprocessing

- XML parsing using BeautifulSoup
- Removal of noise and empty documents
- Tokenization using CountVectorizer
- Stopword removal using English stopwords list

3.2 Topic Modeling (LDA)

Latent Dirichlet Allocation was used to:

- Extract latent topics
- Represent each document as a topic distribution

Parameters:

- Number of topics: 50
- Max iterations: 10
- Learning method: online

LDA transforms each document into a probability distribution over topics.

3.3 Document Clustering (K-Means)

After LDA transformation: Each document is represented in topic-space, K-Means is applied for clustering

Parameters:

- Number of clusters (K): 25
- Initialization: k-means++
- Random state: fixed for reproducibility

4. Results :

```
Cluster 10: 61 docs
Cluster 23: 39 docs
Cluster 20: 34 docs
Cluster 19: 11 docs
Cluster 22: 33 docs
Cluster 17: 41 docs
Cluster 15: 28 docs
Cluster 13: 45 docs
Cluster 12: 22 docs
Cluster 9: 27 docs
Cluster 14: 49 docs
Cluster 2: 51 docs
PS C:\Users\User\Information-Retrieval\HW8>
```



```

PS C:\Users\User\Information-Retrieval\HW8> python partition.py
Cluster 17: 41 docs

Cluster 15: 28 docs

Cluster 13: 45 docs

Cluster 12: 22 docs

Cluster 9: 27 docs

Cluster 14: 49 docs

Cluster 2: 51 docs
PS C:\Users\User\Information-Retrieval\HW8>

```

Tools used were:

```

PS C:\Users\User\Information-Retrieval\HW8> pip install beautifulsoup4 scikit-learn numpy
Requirement already satisfied: numpy in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (2.4.4)
Requirement already satisfied: soupsieve>=1.6.1 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (from beautifulsoup4) (2.8.3)
Requirement already satisfied: typing-extensions>=4.0.0 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (from beautifulsoup4) (4.15.0)
Requirement already satisfied: scipy>=1.10.0 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.3.0 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in c:\users\user\appdata\local\programs\python\python311\lib\site-packages (from scikit-learn) (3.6.0)

[notice] A new release of pip is available: 23.2.1 -> 26.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\User\Information-Retrieval\HW8>

```

6. Conclusion

This assignment successfully demonstrated:

- Topic modeling using LDA
- Document representation in topic space
- Clustering using K-Means
- Meaningful grouping of aeronautical document